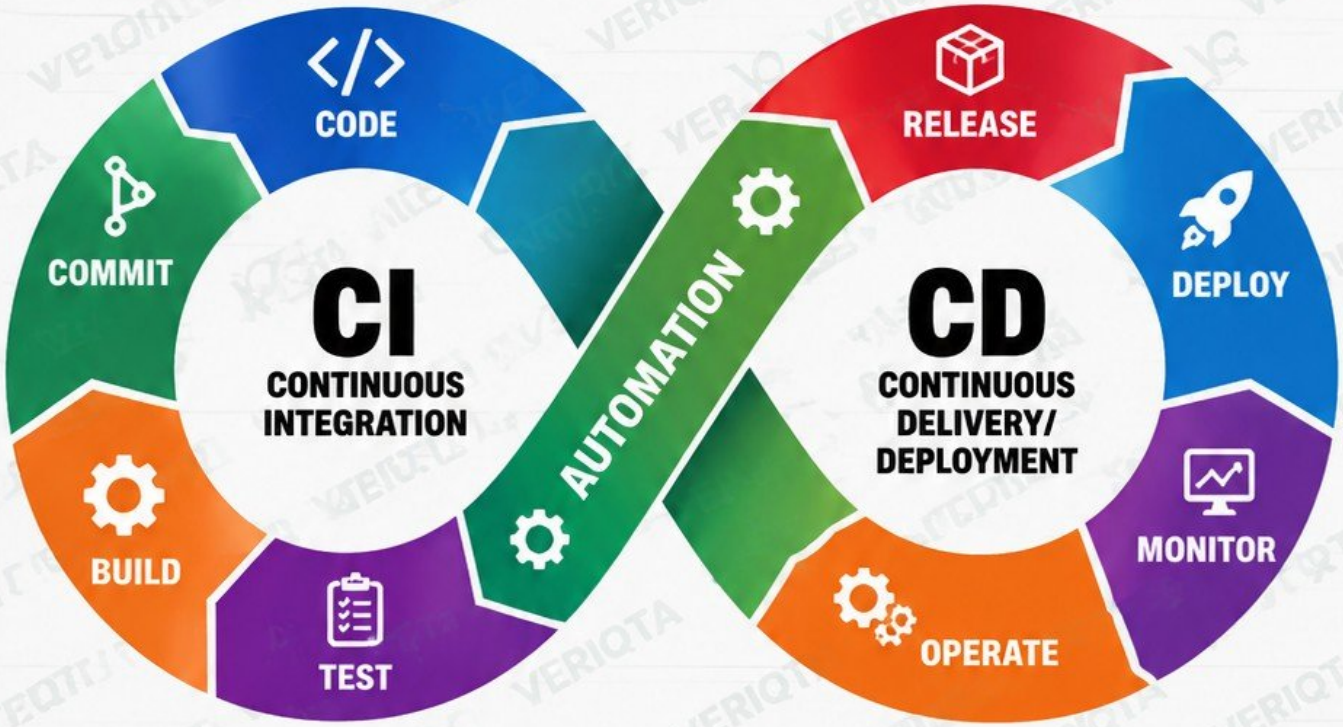
VERIQTA

CI/CD PIPELINE

BUILD • TEST • RELEASE • DEPLOY • OPERATE



FASTER RELEASES	HIGHER QUALITY	LOWER RISK	BETTER COLLABORATION	RELIABLE OPERATIONS

FROM CODE TO PRODUCTION, RELIABLY

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

VERIQTA

CONTENTS



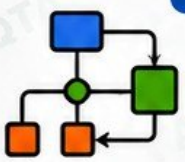
1 INTRODUCTION 03 – 05

1.1 What is CI/CD? 03
1.2 Why CI/CD Matters 04
1.3 Key Benefits 05



2 CORE CONCEPTS 06 – 08

2.1 Continuous Integration (CI) 06
2.2 Continuous Delivery (CD) 07
2.3 Continuous Deployment 08



3 CI/CD PIPELINE STAGES 09 – 12

3.1 Source Stage (Version Control Trigger) 09
3.2 Build Stage 10
3.3 Test Stage 11
3.4 Release Stage 12



4 CI/CD TOOLS 13 – 17

4.1 CI/CD Platforms 13
4.2 Build & Dependency Tools 14
4.3 Artifact Repository Tools 15
4.4 Testing Tools in CI/CD Pipelines 16
4.5 Code Quality & Security Tools (Quality Gates) 17



5 DEPLOYMENT PIPELINES 18 – 22

5.1 Blue/Green Deployments 18
5.2 Canary Releases 19
5.3 Rolling Deployments 20
5.4 Approvals in Deployment Stages (QA Gates) 21
5.5 Environment Promotion Strategy 22



6 ROLES RESPONSIBLE FOR A CI/CD PIPELINE 23 – 26

6.1 Developers 23
6.2 Quality Assurance (QA) Engineers 24
6.3 DevOps Engineers 25
6.4 Operations / SRE (Site Reliability Engineers) 26
6.5 Project Managers / Product Owners 26



7 BEST PRACTICES & NEXT STEPS 27 – 30

7.1 CI/CD Best Practices 27
7.2 Common Challenges and How to Solve Them 28
7.3 Real-World Example Pipeline 29
7.4 Key Takeaways 30

03. UNDERSTANDING CI/CD

Integration, Delivery, Deployment and Release Boundaries

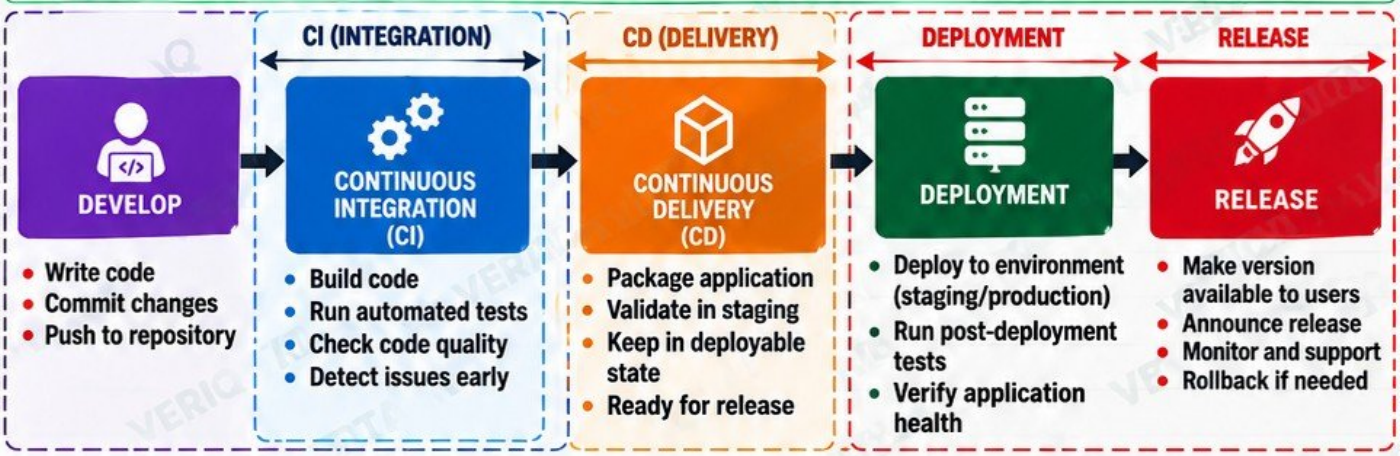
1. What CI/CD Covers

- CI/CD automates the software delivery lifecycle from code commit to production.
- It ensures code is built, tested, validated, packaged and deployed in a repeatable and reliable way.
- It reduces manual work, improves quality and allows faster, safer releases.

Key Terminology

- **CI (Continuous Integration):** Automatically builds and tests code when changes are merged.
- **CD (Continuous Delivery):** Keeps the application in a deployable state and prepares releases.
- **Continuous Deployment:** Automatically deploys to production after passing all checks.
- **Release:** The version of the application delivered to users.

2. End-to-End CI/CD Flow and Boundaries



3. CI/CD Boundaries Explained

Concept	Scope	Production Impact
Continuous Integration (CI)	Builds and tests every change. Focuses on code quality and early feedback.	Does not deploy to production. Detects issues early, reducing risk.
Continuous Delivery (CD)	Automatically prepares a deployable application. Includes packaging, validation and approval gates.	Keeps the application ready for production. Deployment is usually manual or automated later.
Continuous Deployment	Automatically deploys to production after passing all checks.	Changes go to production without manual intervention.
Release	The specific version made available to users.	This is what users receive in production.

4. Key Takeaways

- CI finds problems early.
- CD ensures the application is always deployable.
- Deployment moves the application to an environment.
- A release is the version that users actually get.
- Each stage has a clear boundary and purpose.

5. Real-World Example

A developer pushes code → CI builds and tests it → CD packages and validates it in staging → the team approves → it is deployed to production → a new version is released to users → monitoring ensures it is healthy.

04. CONTINUOUS INTEGRATION (CI)

Triggers, Merge Checks, Build Isolation and Test Gates

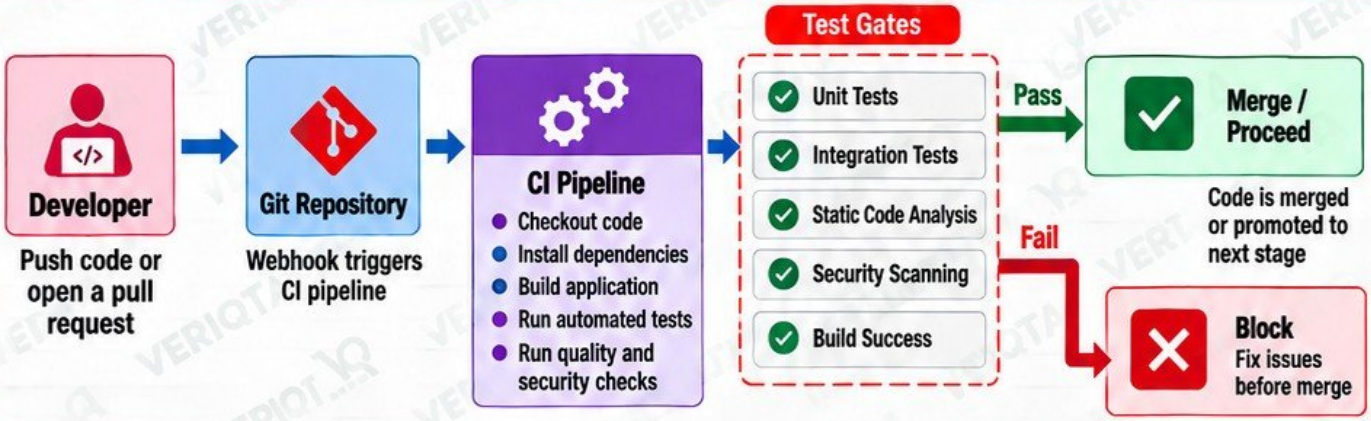
1. What is Continuous Integration (CI)?

- CI is the practice of frequently merging code changes into a shared repository.
- Every change automatically triggers a build and a set of tests.
- It detects integration issues early and keeps the main branch in a healthy state.
- CI is the foundation for Continuous Delivery and Continuous Deployment.

2. Common Triggers for CI

Code push	Pull request	Merge to main	Scheduled build
A developer pushes commits to a branch.	Triggers build and tests for every PR.	Runs full pipeline with stricter checks.	Runs periodic builds for dependency updates.

3. Continuous Integration Flow



4. Merge Checks and Branch Protection

- Require pull requests for changes to main.
- Run mandatory checks (build, test, lint, security).
- Block merge if any check fails.
- Use code review and approvals for critical changes.
- Keep the main branch always deployable.

5. Build Isolation and Reproducibility

- CI runs in isolated environments (e.g., containers or ephemeral runners).
- Ensures consistent builds regardless of the developer machine.
- Prevents "works on my machine" issues.
- Uses clean environments for every run.
- Dependencies are explicitly defined (e.g., package manifests or lock files).

6. Types of Tests in CI

Test Type	Purpose
Unit Tests	Validate individual functions or modules.
Integration Tests	Check interactions between components.
UI Tests	Validate user interface behavior.
API Tests	Validate API endpoints and responses.
Static Analysis	Check code quality, bugs and security issues (e.g., SonarQube, ESLint).
Security Scanning	Detect vulnerabilities in code and dependencies.

7. Production Insights

Common Failure Reasons	• Missing or flaky tests • Dependency conflicts • Environment differences • Long build times
Best Practices	• Keep builds fast and reliable • Run tests in parallel • Fail early on critical issues • Use caching to speed up builds • Maintain a clean and secure build environment
Senior Engineer Tip	A healthy CI pipeline is fast, deterministic and trustworthy. If CI is slow or flaky, teams stop trusting it and start bypassing checks.

05. DELIVERY VS DEPLOYMENT

Approval Gates, Automation and Release Control

1. Key Difference

CONTINUOUS DELIVERY (CD)

- Build, test and package code into a deployable artifact.
- Keeps the application in a deployable state at all times.
- Requires a manual approval to deploy to production.
- Common for most teams that want control.

CONTINUOUS DEPLOYMENT

- Automatically deploys changes to production after passing all checks.
- No manual approval. Deployment is automated.
- Used when tests, monitoring and rollback are highly reliable.
- Common in large-scale cloud-native environments.

2. When to Use Each

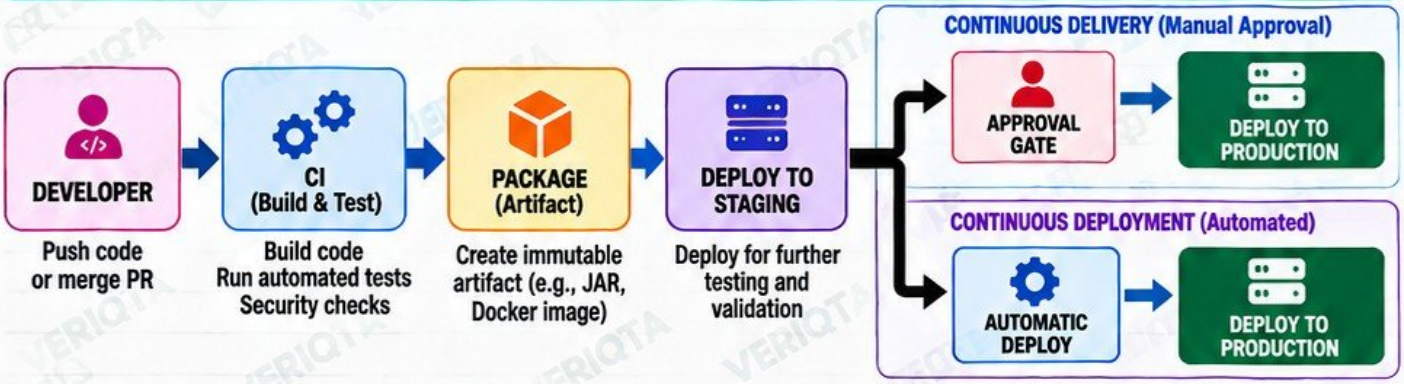
Use Delivery (Manual Approval)

- Regulated environments (e.g., finance, healthcare)
- Need human approval before production
- Higher risk changes
- Feature releases on a schedule
- Teams building confidence in automation

Use Deployment (Fully Automated)

- Strong automated testing and monitoring
- Low risk, frequent changes
- Mature engineering and DevOps practices
- Small, incremental releases
- Fast-moving product teams (e.g., SaaS, tech companies)

3. End-to-End Flow: Delivery vs Deployment



4. Approval Gates

Manual Approval	Product owner, release manager or change advisory board (CAB) approves the release.
Automated Policy Checks	Ensure security, compliance and quality requirements are met.
Environment Validation	Verify staging tests, performance and readiness criteria.
Release Window	Deploy only during approved maintenance or low-traffic periods (if required).

5. Release Control Strategies

Blue-Green	Two environments (current and new). Switch traffic after validation.
Canary	Deploy to a small percentage of users and gradually increase traffic.
Rolling	Replace instances gradually to avoid downtime.
Feature Flags	Deploy code but keep feature disabled until ready.

6. Delivery vs Deployment Comparison

Aspect	Continuous Delivery	Continuous Deployment
Production Deployment	Manual approval	Automatic
Release Control	Human + automated gates	Fully automated
Risk Level	Lower	Higher (requires strong automation)
Speed	Fast but controlled	Fastest
Common Use	Most organizations	Mature cloud-native teams

7. Best Practices

- Keep the application always in a deployable state.
- Automate as many checks as possible.
- Use manual approvals for risky or regulated changes.
- Implement progressive delivery strategies.
- Monitor releases closely and have a rollback plan.

8. Common Risks

- Deploying untested changes to production.
- Weak or missing approval gates.
- Inadequate monitoring after deployment.
- Configuration drift between environments.
- No rollback strategy.

9. Key Takeaway

Continuous Delivery and Continuous Deployment use the same pipeline, but they differ in who approves the release to production. Choose the approach that matches your risk tolerance, regulatory requirements and team maturity.

06. END-TO-END PIPELINE

Source, Build, Test, Scan, Package, Promote and Deploy

1. Complete CI/CD Pipeline Flow

SOURCE
- Commit code
- Open PR
- Trigger pipeline

BUILD
- Compile code
- Install dependencies
- Create artifact
- Run early checks

TEST
- Unit tests
- Integration tests
- UI tests
- Quality checks

SCAN
- Security scan
- Dependency scan
- License check
- Policy checks

PACKAGE
- Create artifact
- Build container
- Push to registry
- Version and sign

PROMOTE
- Move to staging
- Approvals (if needed)
- Run additional tests

DEPLOY
- Deploy to production
- Monitor health
- Rollback if needed

CONTINUOUS FEEDBACK, MONITORING AND IMPROVEMENT

2. Environments and Promotion Flow

DEVELOPMENT
Develop and test new features

STAGING
Validate release in a production-like environment

PRE-PRODUCTION
Final validation, performance and security tests

PRODUCTION
Live environment serving real users

3. What Happens at Each Stage?

Stage	Key Activities	Output
Source	Commit code, open PR, run basic checks	Code in VCS
Build	Compile code, install dependencies	Build artifacts
Test	Run automated tests (unit, integration, UI)	Test results
Scan	Security and dependency scanning	Scan report (pass/fail)
Package	Create artifact or container image	Versioned artifact or image
Promote	Move through stages with approval gates	Approved release candidate
Deploy	Deploy to production, verify and monitor	Application running for users

4. Approval Gates and Controls

Quality Gate	Block deployment if tests, code quality or security scans fail.
Manual Approval	Required for production or high-risk changes.
Policy Enforcement	Check compliance, licenses and security policies.
Environment Validation	Ensure the build is tested in staging or pre-production.
Change Window	Deploy only during approved maintenance windows.
Access Control	Only authorized users can approve or deploy.

5. Common Tools Used in a Pipeline

GitHub · GitLab · Jenkins · Azure DevOps
Docker · Kubernetes · SonarQube · Terraform

6. Key Benefits

- Faster and more frequent releases
- Higher code quality and security
- Reduced manual work and errors
- Clear visibility and traceability
- Reliable and repeatable deployments
- Supports scale and team collaboration

7. Real-World Tip

Start with a simple pipeline that builds, tests and deploys to one environment. Gradually add security scans, multiple stages and approval gates as your system and team grow.

07. SOURCE AND VERSION CONTROL

Git Triggers, Pull Requests and Protected Branches

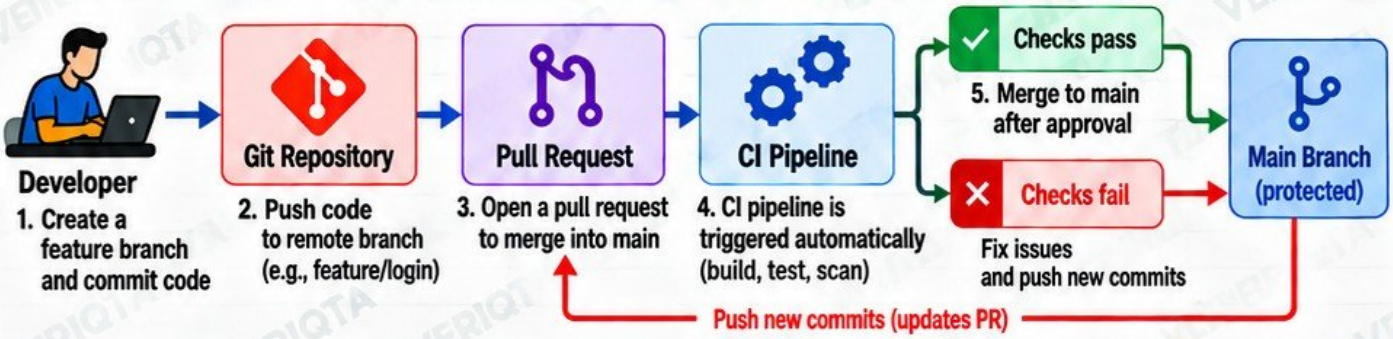
1. What is Source Control?

- Source control (Git) tracks changes to your code and keeps a complete history.
- It allows multiple developers to work together.
- It is the starting point for your CI/CD pipeline.
- Common platforms: GitHub, GitLab, Bitbucket, Azure Repos.

2. Key Concepts

Repository	Stores your project code and history.
Commit	A snapshot of changes.
Branch	A separate line of development.
Merge	Combines changes from one branch to another.
Pull Request (PR)	A request to merge changes with review and checks.
Protected Branch	A branch with rules that restrict direct changes (e.g., main).

3. Typical Git Workflow with Pull Request and CI Trigger



4. Common Triggers for CI

Push	Triggers pipeline when code is pushed to a branch.
Pull Request	Triggers pipeline for every PR to run build and tests.
Merge to Main	Triggers pipeline when code is merged to the main branch.
Tag	Triggers pipeline for a version tag (e.g., v1.0.0).
Scheduled	Runs pipeline at a set time (e.g., nightly builds).
Webhook	Triggers pipeline from external events (e.g., issue created).

5. Protected Branch Rules (Main)

Require pull requests	No direct pushes to main.
Required status checks	All CI checks must pass.
Required reviews	At least one or more approvals.
Restrict who can merge	Only authorized users or teams.
Prevent force pushes	Keeps a clean and safe history.
Require up-to-date branch	Ensures latest changes are tested.

6. Example: GitHub Branch Protection (Concept)

Branch protection rule (main)

- [x] Require a pull request before merging
- [x] Require status checks to pass — CI / Build / Tests
- [x] Require review approvals — At least 1 approval
- [x] Require branches to be up to date
- [x] Restrict who can push to matching branches

7. Best Practices

- Use feature branches for all changes.
- Require pull requests and code reviews.
- Automate CI for every pull request.
- Protect the main branch with required checks.
- Keep commits small and meaningful.
- Use descriptive branch and commit names.
- Monitor and fix failed checks early.

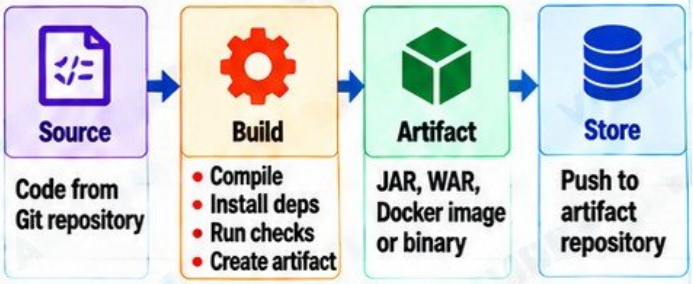
08. BUILD AND DEPENDENCIES

Reproducible Builds, Caching and Dependency Integrity

1. What Happens in the Build Stage?

- Source code is compiled or packaged into an executable artifact (e.g., JAR, Docker image, or binary).
- Dependencies are downloaded and managed.
- The build is reproducible, meaning the same input always produces the same output.
- Build tools use caching to speed up builds.
- The output is an immutable artifact that is passed to the next stages.

2. Build Stage in a CI/CD Pipeline



3. Common Build Tools



Maven (Java) | Gradle (Java) | npm (Node.js) | pip (Python) | .NET (.NET) | Go (Go)

4. Dependency Management

- Build tools download required libraries from public or private repositories.
- Dependencies are defined in files like pom.xml, build.gradle, package.json, requirements.txt, or go.mod.
- Use specific versions to ensure reproducible builds.
- Check for vulnerable or outdated dependencies.

5. Example Build Commands

Maven (Java)
```
mvn clean install
# Compiles code
# Downloads deps
# Runs tests
# Creates artifact
```

Gradle (Java)
```
./gradlew build
# Compiles code
# Runs tests
# Creates artifact
# Uses caching
```

npm (Node.js)
```
npm ci
npm run build
# Installs exact deps
# Builds application
# Uses package-lock.json
```

pip (Python)
```
pip install -r
requirements.txt
# Installs specific
# dependency versions
# for reproducible builds
```

Docker Build
```
docker build -t
myapp:v1.0.0 .
# Builds Docker image
# with application
# and dependencies
```

6. Build Caching

- Caching stores downloaded dependencies and build outputs to speed up future builds.
- Build tools like Maven, Gradle, npm and Docker support caching.
- Common caching locations:
  - Maven: ~/.m2/repository
  - Gradle: ~/.gradle/caches
  - npm: ~/.npm
  - Docker: Layer caching

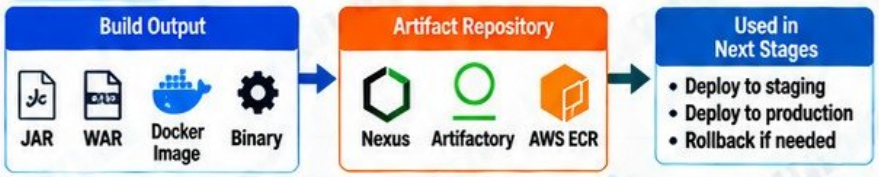
7. Reproducible Builds

- Use version-controlled build files.
- Lock dependency versions (e.g., package-lock.json, pip freeze, go.sum).
- Use the same build environment (e.g., Docker containers).
- Avoid "latest" versions.
- Produce the same artifact from the same source code every time.

8. Dependency Integrity and Security

- Use dependency scanning tools (Snyk, Trivy, OWASP Dependency Check).
- Check for known vulnerabilities in libraries.
- Use private repositories for internal dependencies.
- Verify checksums or signatures when possible.
- Keep dependencies updated and remove unused ones.

9. Build Output and Artifact Repository



10. Key Takeaways

- Build stage compiles code and creates an immutable artifact.
- Use dependency management and caching to speed up builds.
- Ensure builds are reproducible.
- Scan dependencies for vulnerabilities.
- Store artifacts in a repository and promote the same artifact through environments.

09. AUTOMATED TESTING

Unit, Integration, Contract, API and Performance Tests

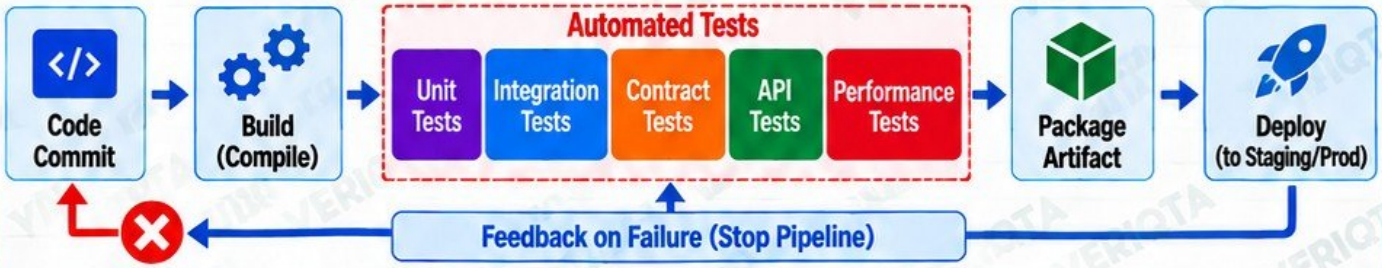
1. Why Automated Testing Matters?

- Detects bugs early in the development cycle.
- Improves code quality and reliability.
- Prevents regressions when new changes are made.
- Allows faster and safer releases.
- Supports continuous integration and deployment.
- Reduces manual testing effort.
- Gives confidence to deploy to production.

2. Types of Automated Tests

Unit Tests	Test individual functions or components in isolation.
Integration Tests	Test interaction between components or services.
Contract Tests	Ensure services communicate using a defined contract.
API Tests	Validate API endpoints, requests, responses and business logic.
Performance Tests	Check load, stress, and scalability under expected and peak traffic.

3. Where Tests Fit in the CI/CD Pipeline



4. Example Tools

JUnit (Java) · pytest (Python) · Jest (JavaScript) · Selenium (UI Tests) · Cypress (UI Tests)

Postman (API Tests) · Rest Assured (API Tests) · Playwright (UI Tests) · JMeter (Performance) · k6 (Performance)

5. What Each Test Validates

Test Type	What It Validates
Unit Tests	Individual functions, methods or modules work as expected.
Integration Tests	Components and services work together correctly.
Contract Tests	Services follow an agreed interface (e.g., API contract).
API Tests	API endpoints return correct responses, status codes and data.
Performance Tests	Application handles expected and peak load (response time, throughput, stability).

6. Test Execution Strategy

Stage	Tests to Run	Purpose
Pull Request (Pre-merge)	Unit tests, static analysis	Fast feedback for developers.
CI Build	Unit, integration, contract tests, API tests	Validate functionality and interfaces.
Pre-Production	Full test suite including performance tests	Ensure readiness for production.
Production (Post-deploy)	Smoke tests, synthetic tests, monitoring	Verify application is working in real environment.

7. Best Practices

- Automate as many tests as possible.
- Keep tests fast, reliable and independent.
- Use realistic test data and environments.
- Run critical tests on every pull request.
- Include both functional and non-functional tests.
- Keep test maintenance as part of the development process.
- Fail the pipeline when essential tests fail.

8. Key Takeaway

A strong automated testing strategy gives confidence, prevents regressions and enables faster, safer releases in your CI/CD pipeline.

10. QUALITY AND SECURITY GATES

SAST, Dependency Scanning, Secrets and Policy Checks

1. What Are Quality and Security Gates?

- Automated checks that ensure code, dependencies and configurations meet quality, security and compliance requirements.
- Run during the CI/CD pipeline.
- Block the pipeline when critical issues are found.
- Help prevent vulnerable or non-compliant code from reaching production.

2. Common Gate Checks

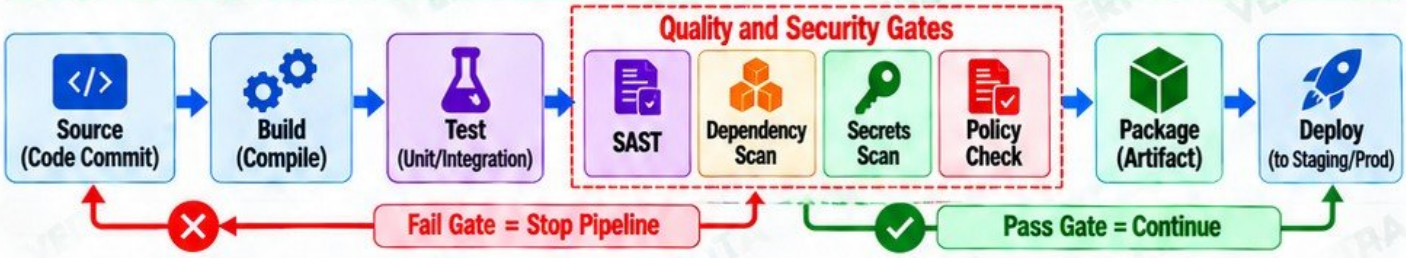
SAST
- Scan source code for vulnerabilities
- Find coding flaws early

Dependency Scanning
- Check open source libraries
- Detect known vulnerabilities (CVEs)

Secrets Scanning
- Find exposed secrets in code
- Prevent API keys, tokens and passwords

Policy Checks
- Enforce security and compliance policies
- Check licenses, configurations and standards

3. Where Security Gates Fit in the Pipeline



4. Key Security Tools

SonarQube (SAST)	Snyk (Dependencies)	Trivy (Containers/Deps)	Checkov (Policy as Code)	Gitleaks (Secrets)
Bandit (Python SAST)	OWASP Dependency-Check	npm audit (Dependencies)	Docker Scout (Containers)	Terrascan (IaC Scan)

5. Example Gate Rules

Gate	Example Rule	Action
SAST	No high or critical vulnerabilities	Fail pipeline
Dependency Scan	No critical CVEs (known vulnerabilities)	Fail pipeline
Secrets Scan	No hardcoded secrets (API keys, passwords)	Fail pipeline
Policy Check	License compliance and approved images	Fail pipeline
Quality Check	Code quality score above threshold	Fail pipeline
Container Scan	No critical vulnerabilities in container image	Fail pipeline

6. Example: Gate Execution in CI/CD



7. Best Practices

- Run security and quality gates on every change.
- Fail fast on critical issues.
- Keep tools and vulnerability databases up to date.
- Use environment-specific policies.
- Scan both application code and infrastructure code.
- Review and manage exceptions properly.

8. Common Failure Examples

- Critical vulnerability found in a dependency (e.g., log4j).
- Hardcoded API key or password detected.
- Use of a disallowed license (e.g., GPL in a proprietary project).
- Container image contains known vulnerabilities.
- Code quality score below the required threshold.

9. Benefits

- Reduce security risks.
- Improve code quality.
- Prevent non-compliant releases.
- Increase confidence in deployments.
- Support regulatory and organizational requirements.

10. Key Takeaway

Quality and security gates ensure only safe, high-quality and compliant code moves forward in your CI/CD pipeline, reducing risk and building trust in your releases.

11. ARTIFACTS AND REGISTRIES

Immutable Artifacts, Image Digests and Promotion

1. What are Artifacts?

- Artifacts are the build outputs produced by your CI pipeline.
- They can be application packages (e.g., JAR, WAR, .NET, npm), container images, Helm charts, or other files.
- Artifacts are immutable, meaning they do not change after they are created.
- Artifacts are stored in registries or artifact repositories for later use.
- The same artifact is promoted through different environments (dev, staging, production).

2. Common Types of Artifacts

JAR (Java)	WAR (Java)	npm (Node.js)	Python Packages	Container Images	Helm Charts
Compiled Java application	Web application archive	JavaScript packages	Python libraries (e.g., wheel)	Docker images (e.g., nginx, myapp)	Kubernetes deployment packages

3. Artifact Flow in a CI/CD Pipeline



e.g. Maven build, Docker build

e.g. JAR, WAR, Docker image

e.g. Nexus, Artifactory, AWS ECR, Docker Hub

Same immutable artifact promoted across environments

4. Artifact Registries

- JFrog Artifactory
- Sonatype Nexus
- AWS ECR
- Docker Hub
- GitHub Container Registry
- Azure Container Registry

5. Immutable Artifacts and Image Digests

- Artifacts should be immutable to ensure consistency and reliability.
- Container images are identified by a unique digest (SHA256), not just a mutable tag like latest.
- Using image digests guarantees you always deploy the exact same image.

Example Docker Image

```
# Using a tag (mutable)
docker pull nginx:latest

# Using a digest (immutable)
docker pull nginx@sha256:3a5f...97b2

Image digest (SHA256):
sha256:3a5f2e6a9c8d4f7e1b2c6...
97b2e9d3f0a4c1d8e6f9a0b3c5d7e1f2
```

6. Artifact Promotion



- The same immutable artifact is promoted across environments.
- No rebuilds between environments.
- Use versioning or digests to track the exact artifact.
- Apply access controls and approval gates for promotion.

7. Example: Pushing and Pulling a Docker Image

Push image to a registry

```
# Login to registry
docker login registry.example.com
# Tag image
docker tag myapp:1.0.0 registry.example.com/myapp:1.0.0
# Push image
docker push registry.example.com/myapp:1.0.0
```

Pull image by digest

```
# Pull immutable image using digest
docker pull registry.example.com/myapp@sha256:3a5f2e6a9c8d4f7e1b2c6...
# Run the image
docker run registry.example.com/myapp@sha256:3a5f2e6a9c8d4f7e1b2c6...
```

8. Best Practices

- Store all build outputs in a centralized artifact registry.
- Use immutable artifacts and image digests.
- Avoid using mutable tags like latest in production.
- Promote the same artifact through all environments.
- Keep a clear versioning and retention strategy.
- Apply access controls, scanning and policy checks.
- Use registry replication and backups for reliability.
- Clean up old artifacts based on retention policies.
- Integrate artifact promotion with approval gates.

12. DEPLOYMENT AND VERIFICATION

Release Execution, Health Checks and Rollback Signals

1. What is Deployment?

- Deployment is the process of releasing a tested and approved artifact to a target environment (e.g., staging or production).
- It makes the new version available to users.
- It is usually automated through a CI/CD pipeline.
- It should include verification to ensure the application is healthy.
- It must have a rollback plan in case of issues.

2. Common Deployment Strategies

Recreate (All at Once)



- Stop old version
- Deploy new version
- Simple but causes downtime

Rolling Deployment



- Replace instances gradually
- No downtime
- Widely used

Blue-Green Deployment



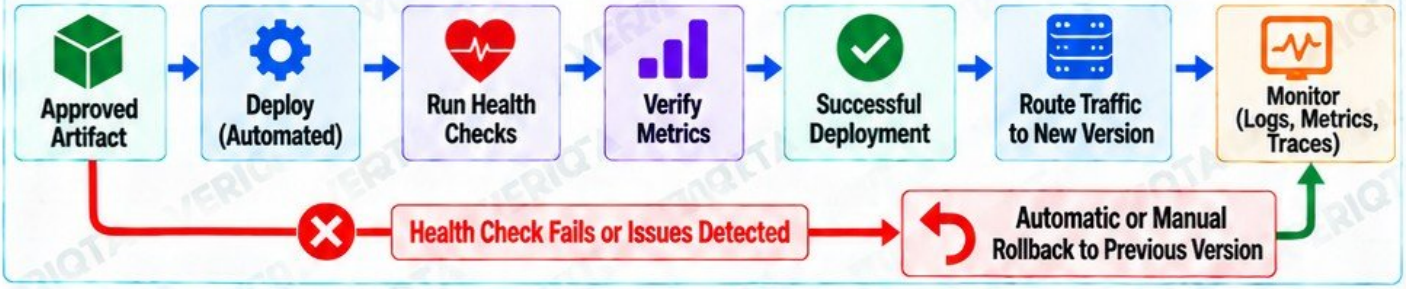
- Two environments (blue and green)
- Switch traffic when healthy
- Quick rollback

Canary Deployment



- Release to a small percentage of users
- Monitor for issues
- Gradually roll out

3. Deployment Flow in a CI/CD Pipeline



4. Health Checks and Verification

Liveness Check

- Confirms the application is running.
- Used to restart unhealthy containers/instances.
- Example: /healthz

Readiness Check

- Confirms the application is ready to serve traffic.
- Used to control traffic flow.
- Example: /ready

Post-Deployment Verification

- Check application logs.
- Monitor key metrics (CPU, memory, error rate, response time).
- Run smoke tests.
- Validate business functionality.

5. Example Health Check Endpoints

```
# Liveness check
curl -f http://localhost:8080/healthz

# Readiness check
curl -f http://localhost:8080/ready

# Check response status
curl -s -o /dev/null -w "%{http_code}\n"
http://localhost:8080/healthz
```

6. Rollback Triggers

- Health checks failing.
- High error rates (e.g., HTTP 5xx).
- Increased latency or failed requests.
- Application crash or container restart loops.
- Failed smoke or integration tests.
- Business metrics showing issues (e.g., failed transactions).

7. Example Rollback Methods

Rollback to Previous Version
Redeploy the last known good artifact.

Rollback Container Image
Use previous image tag or digest.

Revert Infrastructure Change
Undo configuration or infrastructure updates.

Automated Rollback
Triggered by failed health checks or monitoring alerts in the pipeline.

8. Best Practices

- Always automate deployments and verification.
- Use health checks, smoke tests and monitoring.
- Set clear rollback conditions.
- Keep previous versions available.
- Monitor application and business metrics after deployment.
- Use progressive deployment strategies (blue-green or canary) for critical services.
- Test rollback process regularly.

9. Key Takeaway

A successful deployment is not just about releasing the new version, but also verifying it is healthy and being ready to roll back quickly when issues are detected.

13. CI/CD TOOLCHAIN MAP

Tool Logos Organized by Engineering Function

1. Source Control

Git | GitHub | GitLab | Bitbucket

2. CI/CD Orchestration

Jenkins | GitHub Actions | GitLab CI | Azure DevOps | CircleCI

3. Build Tools

Maven | Gradle | npm | Yarn

4. Artifact Repositories

Nexus | Artifactory | AWS ECR | Docker Hub | GitHub Packages

5. Containerization

Docker | Podman | Buildah | Kaniko

6. Container Orchestration

Kubernetes | OpenShift | Amazon EKS | Google GKE | Azure AKS

7. Configuration Management

Ansible | Terraform | Chef | Puppet | SaltStack

8. Infrastructure as Code

Terraform | AWS CloudFormation | Azure Bicep | Pulumi | Crossplane

9. Testing

Selenium | JUnit | Cypress | Postman | k6

10. Security and Compliance

SonarQube | Snyk | Checkov | Trivy | OWASP ZAP

11. Monitoring and Observability

Prometheus | Grafana | ELK Stack | Datadog | New Relic

12. Notification and Collaboration

Slack | Teams | Jira | PagerDuty | Discord

14. CI/CD PLATFORMS

Jenkins, GitHub Actions, GitLab CI/CD, Azure Pipelines

1. Jenkins



Jenkins

- Open source and self-hosted
- Highly customizable
- Large plugin ecosystem
- Suitable for complex CI/CD pipelines

2. GitHub Actions



GitHub Actions

- Native to GitHub
- Event-driven workflows
- Simple YAML configuration
- Deep integration with GitHub ecosystem

3. GitLab CI/CD



GitLab CI/CD

- Built into GitLab
- Single application (dev → deploy)
- Simple and powerful .gitlab-ci.yml
- Great for Docker, Kubernetes and cloud deployments

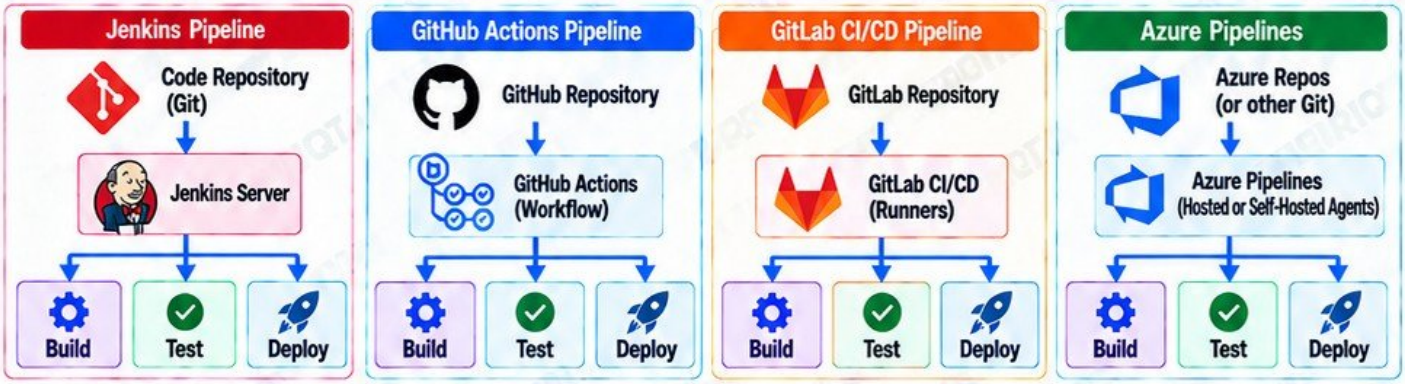
4. Azure Pipelines



Azure Pipelines

- Part of Azure DevOps
- Supports many languages and platforms
- Integrates with Azure and other cloud providers
- Flexible and scalable for enterprise use

5. High Level Architecture



6. Example Pipeline Configuration

Jenkins (Jenkinsfile)

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps { sh 'mvn build' }
    }
    stage('Test') {
      steps { sh 'mvn test' }
    }
    stage('Deploy') {
      steps { sh './deploy.sh' }
    }
  }
}
```

GitHub Actions (.github/workflows/ci.yml)

```
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build
        run: mvn build
      - name: Test
        run: mvn test
      - name: Deploy
        run: ./deploy.sh
```

GitLab CI/CD (.gitlab-ci.yml)

```
stages:
  - build
  - test
  - deploy
build:
  stage: build
  script: mvn build
test:
  stage: test
  script: mvn test
deploy:
  stage: deploy
  script: ./deploy.sh
```

Azure Pipelines (azure-pipelines.yml)

```
trigger:
  - main
pool:
  vmImage: 'ubuntu-latest'
steps:
  - checkout: self
  - script: mvn build
    displayName: 'Build'
  - script: mvn test
    displayName: 'Test'
  - script: ./deploy.sh
    displayName: 'Deploy'
```

7. Key Takeaway

All four platforms automate the same core tasks (build, test and deploy) but each has unique strengths. The best choice depends on your existing ecosystem, team needs and infrastructure.

8. Common Use Cases

- Automating builds, tests and deployments
- Deploying to cloud platforms (AWS, Azure, GCP)
- Building and pushing container images
- Running security scans and quality checks
- Deploying to multiple environments (dev, staging, production)

15. BUILD AND TEST TOOLS

Maven, Gradle, npm, pytest, JUnit, Cypress, Playwright

1. Maven (Java Build Tool)

Maven™

- Builds and manages Java projects
- Manages dependencies
- Uses pom.xml configuration
- Widely used in enterprise applications

pom.xml (example)

```xml
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>app</artifactId>
  <version>1.0.0</version>
</project>
```

2. Gradle (Build Automation)

Gradle

- Builds Java, Kotlin and multi-language projects
- Flexible and extensible
- Faster build performance with incremental builds
- Uses build.gradle configuration

build.gradle (example)

```
plugins {
  id 'java'
}
dependencies {
  implementation 'org.example:
  app:1.0.0'
}
```

3. npm (Node.js Package Manager)

npm

- Manages JavaScript packages and dependencies
- Uses package.json configuration
- Installs, updates and audits packages
- Essential for Node.js applications

package.json (example)

```json
{
  "name": "app",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

4. pytest (Python Testing Framework)

pytest

- Simple and powerful testing framework
- Supports unit, integration and functional tests
- Easy test discovery
- Rich plugin ecosystem

test_example.py

```python
def test_addition():
    assert 2 + 2 == 4

def test_string():
    assert "veriqta".upper() == "VERIQTA"
```

5. JUnit (Java Testing Framework)

JUnit

- Popular testing framework for Java
- Supports unit and integration tests
- Works with Maven and Gradle
- Widely used in enterprise projects

Example JUnit 5 Test

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.
Assertions.*;
class CalculatorTest {
  @Test
  void testAddition() {
    assertEquals(4, 2 + 2);
  }
}
```

6. Cypress (End-to-End Testing)

cy Cypress

- End-to-end testing for web applications
- Easy to set up and use
- Real-time test runner
- Great developer experience

cypress/e2e/example.cy.js

```
describe('Login Test', () => {
  it('should login successfully', () => {
    cy.visit('/login')
    cy.get('#username').type('user')
    cy.get('#password').type('pass')
    cy.get('button').click()
    cy.contains('Welcome')
  })
})
```

7. Playwright (End-to-End Testing)

Playwright

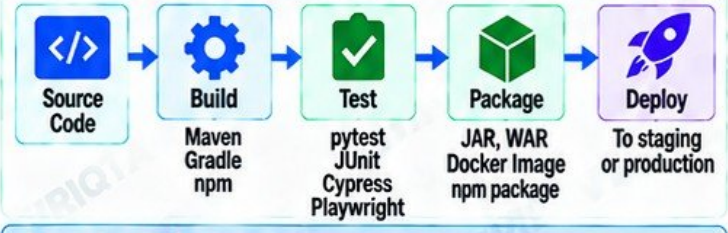
- End-to-end testing for modern web applications
- Supports multiple browsers (Chromium, Firefox, WebKit)
- Reliable and fast
- Supports multiple programming languages (JavaScript, Python, Java, .NET)

Example Playwright Test (JavaScript)

```
conct { test, expect } = require('@playwright/test');

test('has title', async ({ page }) => {
  await page.goto('https://example.comy');
  await expect(page).toHaveTitle(/Example/);
});
```

8. How They Work Together in a Pipeline



9. Key Takeaway

These tools help you build, manage dependencies and test your applications, ensuring high quality and reliable software releases.

16. SECURITY AND ARTIFACT TOOLS

SonarQube, Trivy, Snyk, Harbor, Artifactory

1. SonarQube

SonarQube

- Static code analysis for code quality and security
- Detects bugs, vulnerabilities and code smells
- Supports multiple programming languages
- Integrates with CI/CD pipelines

```
# Example: Run SonarQube analysis
sonar-scanner \
 -Dsonar.projectKey=myapp \
 -Dsonar.sources=.
```

2. Trivy

Trivy

- Scans container images, file systems and configurations
- Detects vulnerabilities in OS packages and dependencies
- Simple, fast and easy to use
- Integrates with CI/CD and registries

```
# Scan a container image
trivy image \
  nginx:latest
```

3. Snyk

Snyk

- Finds and fixes vulnerabilities in dependencies, containers and code
- Provides actionable remediation advice
- Integrates with Git, CI/CD and IDEs
- Supports many languages and package managers

```
# Scan project dependencies
snyk test

# Scan a container image
snyk container test
nginx:latest
```

4. Harbor

Harbor

- Open source container registry with security features
- Supports image signing, scanning and access control
- Role based access control (RBAC)
- Integrates with Kubernetes and CI/CD

```
# Push image to Harbor
docker login harbor.
example.com
docker tag myapp:1.0
harbor.example.com/
myapp:1.0
docker push harbor.
example.com/myapp:1.0
```

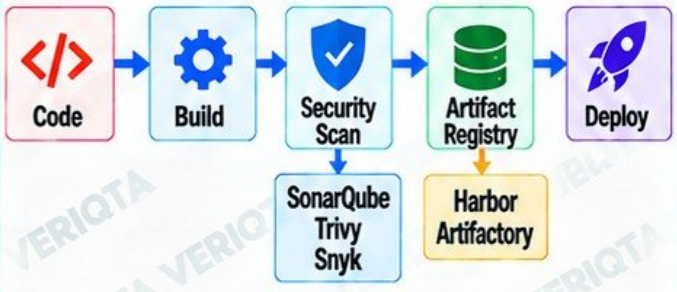
5. Artifactory

JFrog Artifactory

- Universal artifact repository (binaries, packages, container images, Helm charts etc.)
- Supports multiple package formats (Maven, npm, Docker, Python, etc.)
- Access control, replication and high availability
- Widely used in enterprise environments

```
# Deploy a Maven artifact
mvn deploy \
  -DaltDeploymentRepo=
release-repo::default::
https://artifactory.
example.com/artifactory/
maven-releases/
```

6. How They Fit in a CI/CD Pipeline



7. Tool Comparison

Tool	Main Purpose	Key Use Cases
SonarQube	Static code analysis (code quality & security)	Find bugs, vulnerabilities, code smells
Trivy	Vulnerability scanning	Scan container images, file systems, IaC
Snyk	Dependency, container and code scanning	Find and fix vulnerabilities with remediation advice
Harbor	Container registry	Store, secure and manage container images
Artifactory	Universal artifact repository	Store and manage binaries, packages, container images and more

8. Key Benefits

- Improve code quality and security
- Detect vulnerabilities early in the pipeline
- Secure container images and dependencies
- Manage and store artifacts reliably
- Integrate seamlessly with CI/CD tools
- Support multiple languages and package formats

9. Common Use Cases

- Scan code for security issues (SonarQube)
- Scan container images for vulnerabilities (Trivy)
- Check and fix dependency risks (Snyk)
- Store and manage container images (Harbor)
- Manage build artifacts and packages (Artifactory)
- Use together in CI/CD for secure and reliable releases

10. Key Takeaway

SonarQube, Trivy, Snyk, Harbor and Artifactory work together to ensure your code, dependencies, containers and artifacts are secure, high quality and ready for production.

17. DEPLOYMENT AND GITOPS

Argo CD, Flux, Helm, Terraform and Ansible

1. Argo CD

Argo CD

- Declarative GitOps for Kubernetes
- Syncs applications from Git repositories
- Automatic and manual sync options
- Multi-cluster support
- Web UI for visibility

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
spec:
  source:
    repoURL: https://github.com
    path: k8s
    targetRevision: main
```

2. Flux

Flux

- GitOps tool for Kubernetes
- Continuously syncs cluster state with Git
- Supports Helm, Kustomize and raw manifests
- Multi-tenancy support
- Lightweight and highly scalable

```yaml
apiVersion: source.toolkit
kind: GitRepository
metadata:
  name: my-app
spec:
  url: https://github.com
  interval: 1m0s
  ref:
    branch: main
```

3. Helm

Helm

- Kubernetes package manager
- Simplifies application deployment
- Uses charts for repeatable deployments
- Supports versioning and rollback
- Widely used in the Kubernetes ecosystem

```
# Install a chart
helm install my-app \
  ./mychart

# Upgrade
helm upgrade my-app \
  ./mychart

# Rollback
helm rollback my-app 1
```

4. Terraform

Terraform

- Infrastructure as Code (IaC)
- Provisions cloud and on-prem resources
- Multi-cloud support (AWS, Azure, GCP, etc.)
- Declarative and version controlled
- Works well with CI/CD and GitOps

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance"
 "app" {
  ami           = "ami-123456"
  instance_type = "t3.micro"
}
```

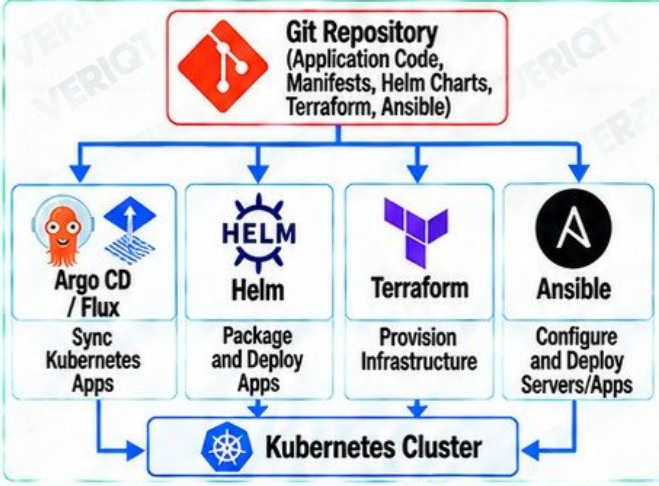
5. Ansible

Ansible

- Configuration management
- Application deployment
- Automates repetitive tasks
- Agentless (uses SSH)
- Simple and powerful yaml syntax

```yaml
- name: Install and start
  nginx
  hosts: web
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
```

6. How They Work Together



7. Example Use Cases

Tool	Common Use Cases
Argo CD	• Deploy and manage Kubernetes applications using Git • Automatic sync and self-healing
Flux	• Keep cluster state in sync with Git • Deploy apps using Helm or Kustomize
Helm	• Deploy and manage Kubernetes applications with charts • Versioning, upgrades and rollbacks
Terraform	• Provision cloud infrastructure (AWS, Azure, GCP) • Manage networking, compute, storage and more
Ansible	• Configure servers and install software • Deploy applications and manage configurations

8. Key Benefits

- Faster and more reliable deployments
- Consistent and reproducible environments
- Infrastructure and applications as code
- Better collaboration and version control
- Supports multi-cloud and hybrid environments
- Improved visibility and automation

9. Common Deployment Flow



10. Key Takeaway

Use Argo CD or Flux for GitOps, Helm to deploy applications, Terraform for infrastructure, and Ansible for configuration to build a complete, automated and scalable deployment strategy.

18. CHOOSING YOUR TOOLCHAIN

Hosting, Integration, Maintenance, Security and Cost

1. Hosting Options

Cloud
- Scalable
- Managed services
- Pay as you go
- Global availability

Self-Hosted
- Full control
- Custom setup
- Potentially lower long term cost
- Requires maintenance

Hybrid
- Flexibility
- Keep sensitive data on-prem
- Use cloud for scale
- More complex setup

Multi-Cloud
- Avoid vendor lock-in
- High availability
- Better resilience
- More complex to manage

2. Integration

Tool Compatibility
- Works well with your stack
- Supports standards (API)
- Easy integration with other tools
- Strong community and plugins

Ecosystem
- Integrates with version control
- Integrates with cloud providers
- Supports monitoring and security tools
- Good marketplace and extensions

3. Maintenance

Operational Effort
- Easy to install and configure
- Regular updates and bug fixes
- Good documentation
- Active community support

Long Term Support
- Stable and reliable
- Clear upgrade path
- Backwards compatibility
- Low operational overhead

4. Security

- Built-in security features
- Regular security updates
- Supports SAST, dependency scanning and image scanning
- Role based access control (RBAC)
- Audit logs and compliance support
- Secrets management integration

5. Cost

- Free and open source options available
- Predictable pricing model
- Cost per user or per resource
- Hidden costs (storage, bandwidth, add-ons)
- Total cost of ownership (TCO)
- Good value for features provided

6. Popular Toolchain Example

GitHub (Code Hosting) → GitHub Actions (CI/CD) → Docker (Build & Package) → Kubernetes (Deploy) → Grafana (Monitor)

7. Comparison Example

Tool	Hosting	Integration	Maintenance	Security	Cost
Jenkins	Self / Cloud	High	Medium	High	Free
GitHub Actions	Cloud	High	Low	High	Free (Paid for more usage)
GitLab CI/CD	Self / Cloud	High	Low	High	Free (Paid tiers)
Azure Pipelines	Cloud	High	Low	High	Free (Paid tiers)

8. Key Questions to Ask

- Where will you host it?
- Does it integrate with your existing tools?
- How much maintenance effort is required?
- What security features does it provide?
- What will it cost now and in the future?
- Does it scale with your needs?

9. Decision Framework

- List your requirements
- Compare tool options
- Evaluate pros and cons
- Consider your team skills
- Factor in long term cost
- Start small and iterate

10. Key Takeaway

- The best toolchain depends on your specific needs.
- Balance hosting, integration, maintenance, security and cost.
- There is no single perfect tool. Choose what fits your team, goals and environment.

19. ROLLING DEPLOYMENTS

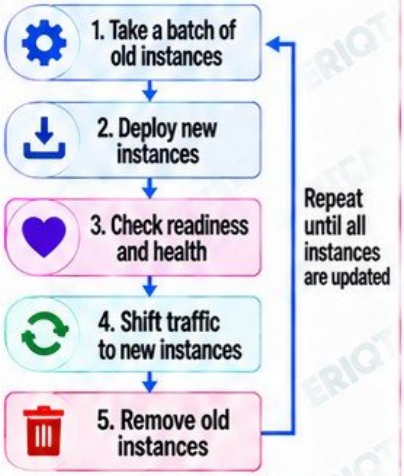
Batch sizing, readiness and unavailable capacity

1. What is a Rolling Deployment?

- Gradually replaces old application instances with new ones.
- Keeps the application available during the update.
- Uses batch sizing and health checks to control the rollout.
- If a problem is detected, the rollout can be paused or rolled back.



2. How It Works



3. Batch Sizing

- Defines how many instances are updated at a time.
- Smaller batches = lower risk but slower rollout.
- Larger batches = faster rollout but higher risk.
- Choose a batch size based on traffic, application type and risk tolerance.



4. Readiness Checks

- New instances must pass readiness checks before receiving traffic.
- Readiness checks verify the application is healthy and can serve requests.
- Common checks: HTTP endpoint, TCP port, application health endpoint, custom checks.
- If a new instance fails readiness, it is not added to the pool.

```yaml
# Example Kubernetes readiness probe
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

5. Unavailable Capacity

- During a rolling update, some instances are temporarily unavailable.
- The unavailable capacity should be small enough to avoid impacting users.
- Use maxUnavailable to control how many instances can be taken down at a time.
- Keep enough capacity to handle traffic during the rollout.
- Consider peak traffic, application performance and SLA requirements.

```yaml
# Example: allow 1 unavailable instance
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

6. Rolling Update Configuration

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    spec:
      containers:
      - name: my-app
        image: my-app:2.0.0
        ports:
        - containerPort: 8080
        readinessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 10
          periodSeconds: 5
```

7. Rolling Deployment Flow



8. Key Takeaway

- Rolling deployments update instances gradually.
- Use appropriate batch sizing based on your risk tolerance.
- Ensure readiness checks prevent unhealthy instances from receiving traffic.
- Maintain enough capacity to handle traffic during the rollout.
- Proper configuration reduces downtime and allows safe, controlled updates.

9. Common Use Cases

- Deploying new application versions.
- Updating configuration or environment variables.
- Rolling out to multiple regions or clusters.
- Minimizing downtime for user-facing applications.

10. Best Practices

- Start with small batch sizes.
- Monitor metrics and error rates during rollout.
- Use automated health checks.
- Set appropriate maxUnavailable and maxSurge.
- Have a rollback plan if issues occur.

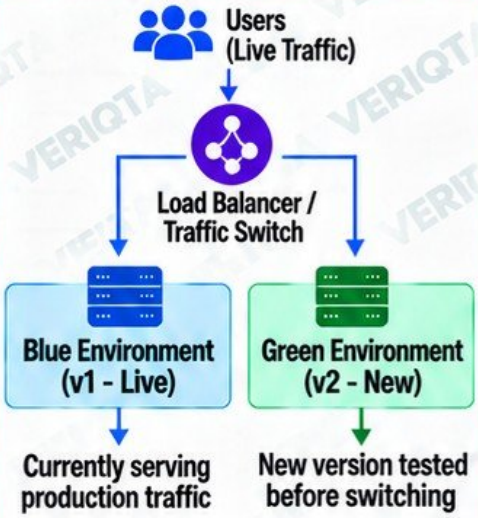
20. BLUE-GREEN DEPLOYMENTS

Environment switching, database compatibility and rollback

1. What is Blue-Green Deployment?

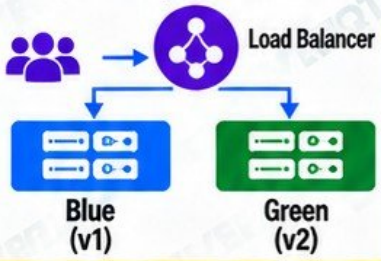
- Runs two identical production environments (Blue and Green).
- Only one environment serves live traffic at a time.
- New version is deployed to the inactive environment.
- Traffic is switched after validation.
- Provides fast and safe rollback by switching back.

2. How It Works



3. Traffic Switching

- Use a load balancer, DNS switch or ingress controller.
- Switch traffic from Blue to Green after validation.
- Switch is usually instant and has minimal downtime.
- Keep the old environment (Blue) for quick rollback.



4. Database Compatibility

- Database changes must be backward compatible.
- New version should work with the existing database schema.
- Use techniques like:
  - Additive changes (add new columns)
  - Feature flags
  - Expand and contract pattern
  - Avoid breaking changes
- Consider running database migrations before or after the switch depending on the change type.

5. Deployment Flow



6. Rollback Process

- If issues are detected, switch traffic back to Blue.
- Rollback is fast because the old environment is still running.
- No need to redeploy the old version.
- Helps minimize downtime and risk to users.



7. Key Benefits

- Minimal downtime
- Fast and safe rollback
- Test new version in production-like environment
- Reduces risk to users
- Works well for web applications and APIs

8. Common Use Cases

- Web applications and APIs
- Microservices
- High availability systems
- Critical applications with low tolerance for downtime
- Regulated environments

9. Best Practices

- Automate testing and validation
- Use automated traffic switching
- Monitor both environments after switch
- Keep Blue environment for a set period
- Plan database changes carefully
- Use feature flags when needed

10. Key Takeaway

- Blue-green deployments use two identical environments to safely release new versions.
- Traffic is switched after validation, and rollback is fast by switching back.
- Database changes must be backward compatible.
- This strategy provides a safe, controlled and low-risk way to deploy applications.

21. CANARY RELEASES

Progressive Traffic, Metrics and Automated Abort Decisions

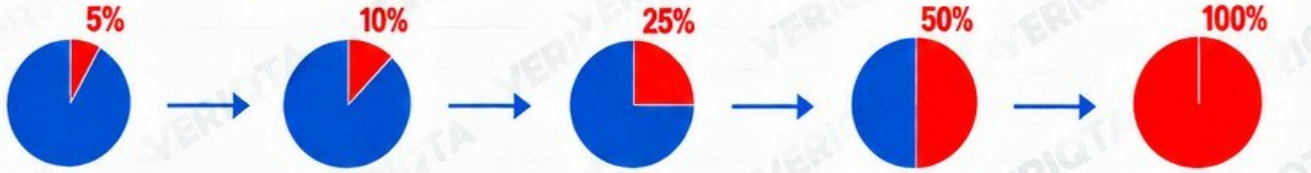
1. What is a Canary Release?

- A canary release gradually sends a small percentage of real user traffic to a new version.
- It helps you detect issues early in a production environment.
- If metrics stay healthy, traffic is increased gradually.
- If issues are detected, traffic is automatically reduced or fully aborted.

2. How Canary Releases Work



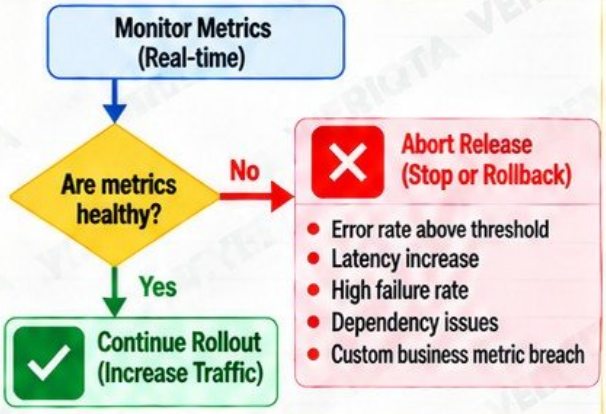
3. Progressive Traffic Increase



4. Key Metrics to Monitor

Error Rate	Increase in 4xx/5xx errors.
Latency	Response time (p50, p95, p99).
Throughput	Requests per second (RPS).
User Experience	Real user monitoring (RUM).
System Health	CPU, memory, and downstream service health.

5. Automated Abort Decisions



6. Example Canary Deployment Flow



7. Best Practices

- Start with a small percentage of traffic.
- Monitor key metrics in real time.
- Set clear thresholds for automatic abort.
- Use automated analysis and alerts.
- Test rollback procedures regularly.
- Keep the canary period long enough to catch real issues.
- Combine with feature flags for extra control.

8. Key Takeaways

- Canary releases reduce risk by gradually exposing new versions to real users.
- Real-time metrics and automated abort decisions protect your users.
- Progressive traffic allows safe validation before full rollout.
- Canary releases help you deliver faster and more reliable software.

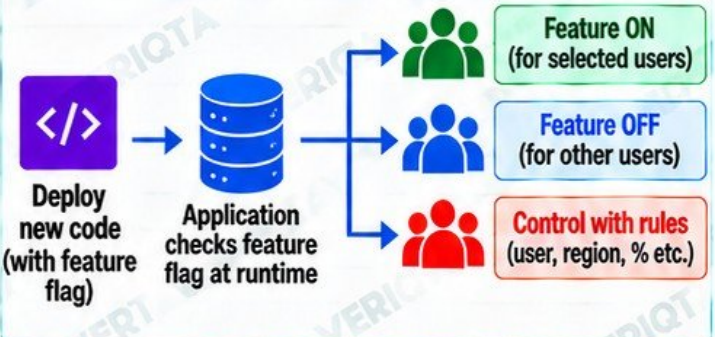
22. FEATURE FLAGS

Separating Feature Activation from Deployment

1. What are Feature Flags?

- Feature flags allow you to deploy code to production without immediately making the feature visible to all users.
- They separate feature activation from deployment.
- You can safely test, control and roll out features to specific users or groups.
- They reduce risk and make releases faster, safer and more flexible.

2. How Feature Flags Work



3. Deployment vs Feature Activation

Deployment (Ships the code)



Code is deployed to production, but the feature is hidden behind a flag.

Feature Activation (Controls visibility)

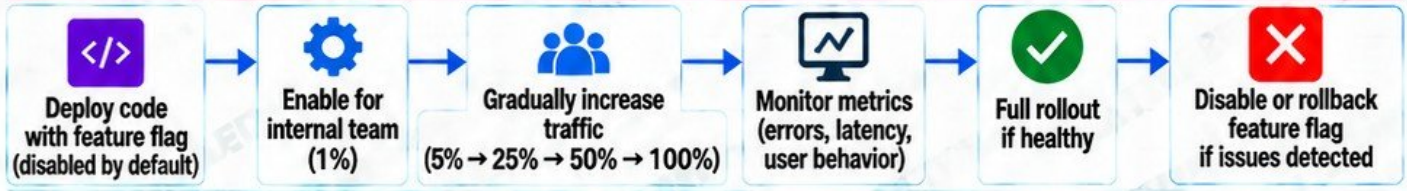


Feature can be turned on or off for specific users, groups or environments without deploying new code.

4. Types of Feature Flags

Release Flags	Gradually roll out features to users (e.g., 5%, 50%, 100%).
Experiment Flags	Run A/B tests to compare different user experiences.
Permission Flags	Enable features for specific users, teams or customers.
Operational Flags	Turn features on/off for operational purposes (e.g., maintenance mode).
Kill Switch Flags	Quickly disable a feature if issues are detected.

5. Example: Feature Flag Rollout Flow



6. Common Use Cases

- Gradual feature rollout to a percentage of users.
- A/B testing different versions of a feature.
- Enabling features for specific regions or customers.
- Testing new features in production safely.
- Quickly disabling a problematic feature (kill switch).
- Running previews for internal teams before full release.

7. Example Implementation (Concept)

```
# Check feature flag at runtime
if feature_flag("new_checkout"):
    return new_checkout_flow()
else:
    return old_checkout_flow()
```

The same code is deployed, but the feature is controlled by a flag, allowing you to enable or disable it without a new deployment.

8. Best Practices

- Keep flags granular and targeted.
- Use clear naming and documentation.
- Set an expiry date for temporary flags.
- Monitor metrics and user feedback.
- Clean up old flags to avoid technical debt.
- Use feature flags as part of a gradual and controlled release strategy.

9. Key Takeaways



- Feature flags separate feature activation from deployment.
- They enable safer, faster and more flexible releases.
- You can control who sees a feature and when.
- They reduce risk and make it easy to rollback.
- Always monitor and clean up old flags.

23. RELEASE APPROVALS

Change controls, separation of duties and audit evidence

1. What are Release Approvals?

- A controlled process to approve changes before they go to production.
- Ensure the right people review and authorize releases.
- Reduce risk, prevent unauthorized changes and maintain stability.
- Enforce policies, compliance and separation of duties.
- Create audit evidence for accountability and traceability.

2. Why Release Approvals Matter?

- Reduce Risk
- Ensure Compliance
- Enforce Separation of Duties
- Prevent Unauthorized Changes
- Maintain Stability
- Provide Audit Evidence
- Build Accountability
- Meet Regulatory Requirements

3. Example Release Approval Flow

1. **Change Request** (e.g., new release)
2. **Automated Checks** (tests, security, policy)
3. **Approval by Authorized Personnel** (e.g., change board)
4. **Deploy to Production** (approved release)
5. **Record Audit Evidence** (approval logs, change records)

4. Who Approves What?

Change Type	Typical Approver(s)
Standard Change (low risk)	Automated approval or designated team lead
Normal Change (medium risk)	Change advisory board (CAB) or release manager
High Risk / Major Change	Senior management + change advisory board
Emergency Change	Designated emergency approver (with post-review)

5. Separation of Duties

- **Developers** — Build and submit changes (cannot approve)
- **Reviewers** — Review and approve changes (not the same person who developed)
- **Release Manager** — Execute approved releases (follows the process)
- **Auditors / Compliance** — Verify process and audit evidence (independent)

6. Common Approval Controls

- Required reviews and approvals.
- Automated policy and security checks.
- Separation of duties (no self-approval).
- Time-based approval windows.
- Multi-level approvals for high-risk changes.
- Mandatory audit logging and change records.

7. Example Approval Policy (Concept)

```yaml
approval-policy:
  environments:
    staging:
      required_approvals: 1
      approvers: ["team-lead"]
    production:
      required_approvals: 2
      approvers: ["release-manager", "cab"]
```

8. Best Practices

- Define clear approval criteria.
- Enforce separation of duties.
- Use automated checks before approval.
- Document and store all approvals.
- Keep the process simple and consistent.
- Review and improve the process regularly.
- Ensure emergency change procedures exist.

9. Key Takeaways

- Release approvals reduce risk and ensure stable deployments.
- They enforce separation of duties and prevent unauthorized changes.
- They provide audit evidence for compliance and accountability.
- A clear approval process helps you deliver changes safely and reliably.

24. ROLLBACK AND RECOVERY

Application rollback, roll-forward and database migration risks

1. What is Rollback and Recovery?

- Rollback returns the system to a previous working version when issues are detected.
- Roll-forward applies a new fixed version instead of going back.
- Recovery restores the service to a stable and healthy state.
- It reduces downtime, limits business impact and keeps users safe.
- A well-defined rollback and recovery plan is essential for reliable deployments.

2. Rollback vs Roll-Forward

Rollback (Go Back)



- Return to a previous stable version.
- Useful for quick recovery from issues.
- Requires keeping previous versions available.
- May lose recent changes if not carefully managed.

Roll-Forward (Go Forward)



- Deploy a new fixed version with the issue resolved.
- Better when rollback is not possible or safe.
- Keeps the latest features.
- Reduces risk when database changes are not backward compatible.

3. Application Rollback Flow

Detect Issue (monitoring, alerts or user reports) → Validate Impact (confirm root cause) → Initiate Rollback (redeploy previous version) → Verify Functionality (run smoke tests and key checks) → Monitor Stability (check metrics and user feedback) → Confirm Recovery (system operating normally)

4. Database Migration Risks



- Schema changes may not be backward compatible.
- Rolling back database changes can be difficult or impossible.
- Data loss or corruption if migrations fail.
- Application and database version mismatch.
- Large migrations can cause performance issues.
- Some changes require roll-forward instead of rollback.

5. Strategies to Reduce Database Risks

- Use backward compatible schema changes.
- Test migrations in a staging environment.
- Take database backups before applying changes.
- Use feature flags and phased rollouts.
- Keep migration scripts versioned and documented.
- Have a clear roll-forward plan if rollback is not possible.

6. Rollback and Recovery Decision Flow

Issue Detected → Is it safe to rollback?

- Yes → Rollback to previous version → Verify and Monitor
- No → Deploy fixed version (roll-forward) → Verify and Monitor

→ Service Recovered

7. Best Practices

- Keep previous versions readily available.
- Automate rollback processes where possible.
- Test rollback and recovery regularly.
- Monitor key metrics after recovery.
- Document procedures and responsibilities.
- Consider database rollback limitations before deployment.

8. Common Tools

- Kubernetes (rollbacks)
- Helm (release history)
- Database Backups
- GitLab (releases)
- Argo CD (rollbacks)
- AWS (blue/green, rollback)

9. Key Takeaways



- Rollback and recovery reduce risk and downtime.
- Roll-forward is often safer for database changes.
- Always test and automate your recovery procedures.
- Database migrations require extra planning and careful handling.
- A clear recovery plan keeps your service reliable and your users safe.

25. PIPELINE SECURITY

Runner isolation, least privilege, OIDC and secret protection

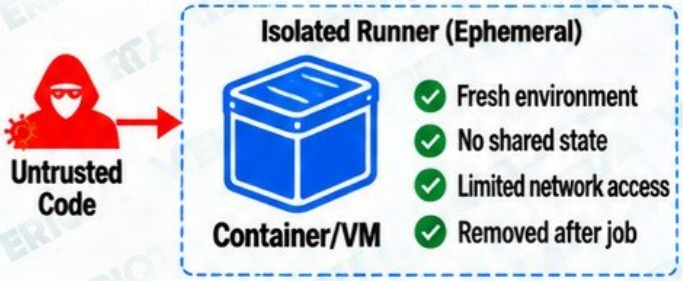
1. What is Pipeline Security?

- Protects your CI/CD pipeline, runners, secrets, identities and infrastructure from attacks.
- Prevents malicious code, unauthorized access and supply chain risks.
- Ensures runners are isolated and have only the permissions they need.
- Uses modern identity methods like OIDC instead of long-lived keys.
- Keeps secrets safe throughout the pipeline and in external systems.

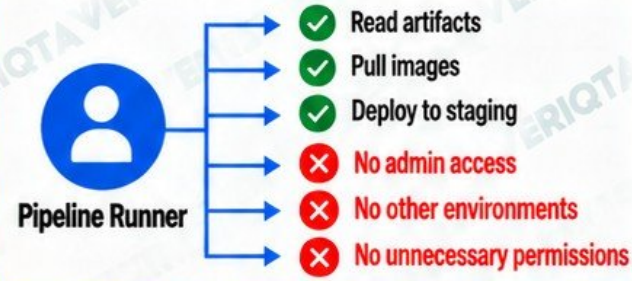
2. Why Pipeline Security Matters?

- Prevents supply chain attacks
- Protects sensitive secrets
- Reduces unauthorized access
- Ensures compliance and auditability
- Limits damage if a runner is compromised
- Keeps cloud resources secure
- Builds a more reliable and trustworthy release process
- Protects your users and business

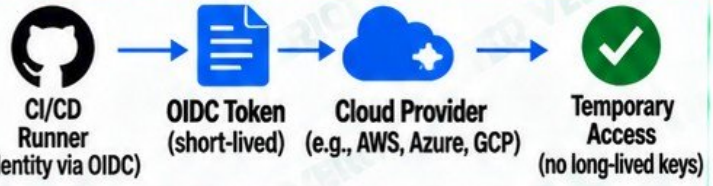
3. Runner Isolation



4. Least Privilege Access



5. OIDC (OpenID Connect)



- Uses short-lived tokens instead of permanent credentials.
- Authenticates the pipeline runner securely.
- Reduces risk of leaked long-lived keys.
- Supports fine-grained, time-bound permissions.

6. Secret Protection

- Store secrets in a secure secret manager (e.g., AWS Secrets Manager, Azure Key Vault, HashiCorp Vault).
- Use environment secrets and mask them in logs.
- Rotate secrets regularly.
- Never hardcode secrets in code or pipeline configuration.

7. Common Risks

- Malicious code in pull requests.
- Compromised or shared runners.
- Excessive permissions and broad access.
- Leaked secrets in logs or artifacts.
- Long-lived cloud credentials.
- Access to production environments.

8. Best Practices

- Use isolated, ephemeral runners.
- Apply least privilege permissions.
- Use OIDC for cloud access.
- Store secrets in a dedicated secret manager.
- Mask secrets in logs.
- Rotate credentials regularly.
- Scan pipeline code and dependencies.
- Monitor and audit pipeline activity.

9. Key Takeaways

- Secure your pipeline like a production system.
- Isolate runners and use least privilege.
- Prefer OIDC over long-lived keys.
- Protect secrets at every stage.
- Monitor, audit and continuously improve.

26. SUPPLY CHAIN INTEGRITY

SBOMs, signing, provenance and artifact verification

1. What is Supply Chain Integrity?

- Ensures the software you build and deploy comes from trusted and verified sources.
- Helps you know exactly what is in your software (components, libraries and dependencies).
- Uses SBOMs, signing and provenance to detect tampering or malicious components.
- Reduces the risk of supply chain attacks and compromised dependencies.
- Builds trust, compliance and a more secure software delivery pipeline.

2. Key Components

SBOMs
A detailed list of all software components, dependencies and versions in your build.

Signing
Cryptographically sign build artifacts to prove they are from a trusted source and not tampered with.

Provenance
Records how, where and by whom the artifact was built (build metadata and pipeline details).

Artifact Verification
Verify signatures and provenance before deploying or using artifacts.

3. How It Works (End to End)

Source Code (with trusted dependencies) → Build (generates SBOM) → Sign (cryptographic signature) → Attach Provenance (build metadata) → Store in Registry (signed artifact +SBOM + provenance) → Verify Before Use (check signature, provenance and SBOM)

4. Example: Artifact Verification Flow



5. Tools and Standards

CycloneDX	Generate and consume SBOMs.
SPDX	Industry standard for SBOMs.
Sigstore	Signing and verification (cosign).
in-toto	Supply chain provenance.
SLSA	Framework for secure build pipelines.

6. Risks Without Supply Chain Integrity

- Malicious or compromised dependencies.
- Tampered build artifacts.
- No visibility into what's in your software.
- Difficulty detecting and responding to supply chain attacks.
- Compliance and audit failures.

7. Best Practices

- Generate SBOMs for every build.
- Sign all build artifacts.
- Record and store provenance.
- Verify artifacts before deployment.
- Use trusted and pinned dependencies.
- Scan SBOMs for known vulnerabilities.
- Automate verification in your pipeline.

8. Key Takeaways

- Know what's in your software (SBOMs).
- Trust the source (signing and provenance).
- Verify before you use (artifact verification).
- It reduces risk and prevents supply chain attacks.
- It builds a more secure, reliable and compliant delivery pipeline.

27. CI/CD OBSERVABILITY

Lead time, failure rates, deployment health and SLO signals

1. What is CI/CD Observability?

- Gives visibility into the health and performance of your delivery pipeline.
- Helps you measure key metrics like lead time, change failure rate and deployment success.
- Shows you where bottlenecks, failures and risks are happening.
- Allows you to detect issues early and keep deliveries reliable and fast.
- Supports continuous improvement with data-driven decisions.

2. Key CI/CD Metrics (DORA)

Lead Time for Changes
Time from code commit to production.

Change Failure Rate
Percentage of deployments that cause a failure.

Deployment Frequency
How often you deploy to production.

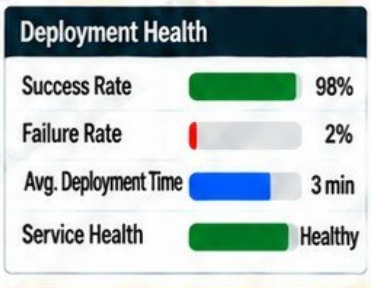
Mean Time to Recovery (MTTR)
Time to restore service after a failure.

3. CI/CD Observability Flow



4. Deployment Health Signals

- Successful deployments
- Failed or rolled back deployments
- Deployment duration
- Error rates after deployment
- User impact (latency, availability, errors)
- Resource usage (CPU, memory, etc.)



5. Common Tools

Prometheus · Grafana · ELK Stack · Datadog · New Relic · OpenTelemetry

6. Example CI/CD Observability Dashboard



7. Best Practices

- Track DORA metrics regularly.
- Monitor deployment health and user impact.
- Set alerts for failed deployments and anomalies.
- Visualize trends with clear dashboards.
- Use data to identify bottlenecks and improve.
- Keep SLOs and reliability goals in focus.

8. Common Issues

- Lack of visibility into pipeline performance.
- Slow or unreliable deployments.
- High change failure rates.
- Missing or noisy alerts.
- No clear ownership of failures.
- Not using data to drive improvements.

9. Key Takeaways

- CI/CD observability helps you deliver faster and safer.
- Track lead time, failure rates, deployment health and SLO signals.
- Use the right tools and dashboards.
- Detect issues early and reduce risk.
- Data-driven improvement leads to a more reliable and efficient delivery pipeline.

28. PIPELINE TROUBLESHOOTING

Build failures, runner exhaustion, flaky tests and stalled deployments

1. What is Pipeline Troubleshooting?

- Identifies and fixes issues that cause pipeline failures or delays.
- Helps keep builds, tests and deployments reliable and fast.
- Reduces wasted time and infrastructure costs.
- Improves developer productivity and confidence.
- Ensures smooth and consistent software delivery.

2. Common Pipeline Issues

Build Failures
Compilation errors, missing dependencies or misconfigurations.

Runner Exhaustion
Not enough runners, slow runners or resource limits.

Flaky Tests
Tests that pass sometimes and fail randomly.

Stalled Deployments
Deployments stuck due to approvals, timeouts or environment issues.

Configuration Issues
Wrong pipeline settings or environment variables.

Dependency Problems
Missing or incompatible packages and tools.

Network Issues
Registry, API or service connectivity problems.

Timeouts
Jobs taking too long and hitting time limits.

3. Troubleshooting Flow

Identify the Issue
Check logs, error messages and pipeline status.

→ **Gather Information**
Review job logs, artifacts and metrics.

→ **Find Root Cause**
Check configuration, dependencies, resources and external services.

→ **Apply a Fix**
Update code, configuration or infrastructure.

→ **Re-run and Verify**
Trigger the pipeline again and confirm the issue is resolved.

4. Example: Build Failure (Logs)

```
Run mvn clean package
[ERROR] Failed to execute goal
  on project demo-service
[ERROR] Could not resolve dependencies
  for project demo-service
[ERROR] The following artifacts
  could not be resolved:
    org.example:utils:jar:1.0.0
[ERROR] -> Check repository, version
  and network connectivity.
```

5. Runner Exhaustion Troubleshooting

- Check runner capacity and current usage.
- Look for stuck or long-running jobs.
- Scale up runners or use autoscaling.
- Optimize job duration and caching.
- Clean up unused or idle runners.

6. Flaky Tests Troubleshooting

- Check test logs and error patterns.
- Identify timing or environment issues.
- Use retries with caution.
- Stabilize test data and external dependencies.
- Run tests in isolated, consistent environments.
- Fix the root cause instead of only retrying.

7. Stalled Deployment Troubleshooting

- Check deployment logs and status.
- Verify environment health and resource availability.
- Look for pending approvals or manual steps.
- Check timeouts and external service dependencies.
- Roll back if necessary and re-deploy after fixing the issue.

8. Common Tools

GitHub Actions · GitLab CI/CD · Jenkins · Argo CD · Kubernetes · Prometheus · Grafana

9. Best Practices

- Keep logs and artifacts for debugging.
- Use clear and informative error messages.
- Monitor runner usage and job duration.
- Avoid flaky tests and maintain test stability.
- Set reasonable timeouts and retry policies.
- Document common issues and solutions.

10. Key Takeaways

- Most pipeline issues can be solved by checking logs, configuration, resources and external dependencies.
- Runner exhaustion, flaky tests and stalled deployments are common but avoidable.
- A systematic approach helps you find root cause quickly.
- Fix the underlying issue, not just the symptom.
- Reliable pipelines keep your team productive and your deployments on track.

11. Common Error Messages

- **"Command not found"** Missing tool or incorrect image.
- **"Dependency not found"** Wrong version or repository issue.
- **"Runner is busy"** Not enough available runners.
- **"Job timed out"** Pipeline step taking too long.
- **"Deployment stuck"** Waiting for approval or external service.

29. PRODUCTION INCIDENT STUDY

A failed release investigated through evidence and recovery

1. Incident Overview

- A new release was deployed to production and caused service degradation.
- Users experienced high error rates, slow response times and some outages.
- This incident study shows how to investigate the issue using evidence and recover the service.
- We use real-world steps, logs, metrics and system data to find the root cause.
- The goal is to learn, fix the issue and prevent it from happening again.

2. Impact

Users Affected	Error Rate	Response Time	Duration
~45%	12% ↑	8s ↑ (normal: 300ms)	37 mins

3. Timeline (What Happened)

10:00	10:05	10:10	10:15	10:37
Release deployed	Error rate increased	User reports started	Rollback initiated	Service recovered

4. Investigation Steps

1. Check alerts and monitoring (identify the symptoms)
2. Review deployment changes (what was released)
3. Analyze logs and metrics (find evidence)
4. Identify the root cause (confirm why it happened)
5. Implement the fix and recover (restore the service)
6. Document lessons and prevention (avoid recurrence)

5. Evidence Collected

Logs	Error messages in application logs Increased timeouts Database connection errors
Metrics	Spike in error rate High CPU and memory usage Increased database latency
Deployment Data	New version deployed at 10:00 Configuration change detected New dependency version
Traces	Slow database queries Increased latency in downstream service Higher than normal request retries
User Reports	Users unable to log in Intermittent 500 errors Slow page loads

6. Root Cause

A database schema change introduced in the new release caused slow queries and connection exhaustion.

- The application was not fully backward compatible with the database change.
- Connection pool limits were reached, causing timeouts.
- Retry logic from the application increased the load on the database.
- This led to cascading failures and service degradation.

7. Recovery Actions

Rolled back to previous stable version → Restored database connections → Monitored system health → Confirmed service recovery

8. Results

- Service fully recovered in 37 minutes.
- Error rate returned to normal.
- Response time back to 300ms.
- All user-facing functionality restored.
- No data loss.
- Incident documented for future prevention.

9. Key Lessons

- Test database changes in a production-like environment.
- Use feature flags for risky changes.
- Have a rollback plan ready.
- Monitor key metrics immediately after deployment.
- Keep changes backward compatible where possible.
- Conduct regular incident response drills.

10. Preventive Measures

- Add automated checks for schema changes.
- Use gradual rollouts and canary releases.
- Improve database connection pool limits.
- Set alerts for error rate, latency and DB health.
- Add load testing for major releases.
- Document and review changes before deployment.
- Continue to improve monitoring and on-call response.

11. Key Takeaways

- Investigate using evidence, not assumptions.
- Quick detection and rollback reduce impact.
- Automation, monitoring and testing help prevent future incidents.
- Learn from incidents and improve your release process.

30. SENIOR ENGINEER REVIEW

Architecture decisions, release-readiness checks and scenario questions

1. What This Review Covers

- Evaluates your ability to make sound architecture and delivery decisions.
- Confirms you understand release-readiness checks and operational risks.
- Tests your problem-solving skills with real-world scenarios.
- Brings together everything from the previous topics into practical, senior-level thinking.
- Helps you identify areas to improve and build confidence for real engineering roles.

2. Architecture Decisions

- Choose the right architecture for the problem (e.g., monolith vs microservices).
- Design for scalability, reliability and maintainability.
- Consider data storage, caching and database design.
- Evaluate trade-offs (cost, complexity, performance, team skills).
- Plan for security, compliance and least privilege.
- Design for observability and operability.
- Consider team workflows and developer productivity.
- Make decisions that balance business value and technical sustainability.

3. Release-Readiness Checks

- All tests passing (unit, integration, end-to-end).
- Security scans completed (no critical vulnerabilities).
- Dependencies and infrastructure ready.
- Database changes reviewed and tested.
- Rollout plan and rollback plan in place.
- Monitoring, alerts and SLOs configured.
- Approvals completed (change control).
- Go / No-Go decision based on evidence.

4. Scenario Questions (Examples)

- Q A new release causes high error rates. How would you investigate and resolve it?
- Q A database migration is taking longer than expected. What are your options?
- Q You need to roll out a high-risk feature. How would you reduce the risk?
- Q A critical dependency is showing instability. How would you handle the release?
- Q Your CI/CD pipeline is slow and frequently fails. What steps would you take to improve it?
- Q How would you design a system to handle 10x more traffic?

5. Evaluation Areas

- Technical Decision Making
- Problem Solving
- Risk Assessment
- Operational Readiness
- Security and Compliance
- System Design
- Communication and Justification
- Practical Experience Mindset

6. Best Practices

- Use evidence and data to support your decisions.
- Consider multiple options and explain the trade-offs.
- Think about security, reliability, cost and maintainability.
- Demonstrate clear and structured problem-solving.
- Show how you would monitor, test and validate changes.
- Be ready to explain your reasoning and assumptions.

7. Common Mistakes

- Making decisions without considering trade-offs.
- Ignoring security, monitoring or rollback plans.
- Overlooking operational and business impact.
- Not communicating the reasoning behind your decisions.
- Focusing only on happy path scenarios.

8. Key Takeaways

- A senior engineer makes informed, balanced and practical decisions.
- Release-readiness is about reducing risk and ensuring safe delivery.
- Real-world scenarios test your ability to apply knowledge, not just recall theory.
- Continuous learning and reflection help you grow as a senior engineer.